UNIVERSIDAD AUTÓNOMA DEL PERÚ

ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE

INFORME DEL MVP

EXAMEN PARCIAL INTEGRADO – SEMANA 8

Desarrollo Web Full Stack FrontEnd
Laboratorio de Experiencias Web Inmersivas

SIDAA Finanzas

Gestión de ingresos gastos y análisis financiero

Primer hito del proyecto semestral
Implementación frontend y validación técnica

Semestre académico 2026-2

Resumen ejecutivo

SIDAA Finanzas permite organizar ingresos y gastos, consultar movimientos y comprender la evolución del saldo. El problema abordado es la dispersión de registros y la dificultad para comparar periodos y categorías. El MVP integra React y Vite con una API Express y almacenamiento en el servidor. Incluye registro, acceso, operaciones financieras, búsqueda, eliminación, resúmenes, gráficos, temas visuales y monedas de referencia. La migración conserva el diseño y utiliza componentes, props, estado y efectos con limpieza. La validación principal se realizó sobre la aplicación alojada en Render mediante una cuenta exclusiva y datos de prueba identificados. Se registraron un ingreso de 1000 USD y un gasto de 250 USD; el saldo de 750 USD permaneció tras recargar e iniciar sesión desde otro contexto de navegador. Los 18 casos de producción se aprobaron y los dos movimientos se eliminaron al finalizar. Como complemento, las 15 pruebas automatizadas locales verificaron lógica y limpieza de recursos. Nueve combinaciones de vista y tamaño no presentaron desbordamiento horizontal global, aunque se confirmó un recorte del importe en el resumen. Lighthouse público obtuvo 76 en rendimiento, 93 en accesibilidad y 100 en buenas prácticas y SEO. La continuidad prioriza corregir contraste, semántica, recorte y carga de recursos antes de ampliar funciones.

1. Descripción del proyecto

Tabla 1. Descripción del proyecto

Elemento	Descripción del equipo
Nombre del proyecto	SIDAA Finanzas
Problema o necesidad	Los registros dispersos dificultan conocer el saldo disponible, identificar gastos y comparar la evolución mensual.
Público objetivo	Personas que gestionan sus finanzas personales y necesitan un registro sencillo de ingresos y gastos.
Objetivo general	Desarrollar un MVP web que centralice movimientos financieros y presente información comprensible para su seguimiento.
Propuesta de valor	Registro, búsqueda, indicadores y gráficos en una sola aplicación adaptable, con preferencias visuales y monedas de referencia.

2. Alcance del MVP – Hito de Semana 8

2.1 Funcionalidades implementadas

El flujo autenticado se comprobó en https://finanzasmainupgraded.onrender.com/ con una cuenta exclusiva. Se verificó persistencia mediante recarga y nuevo inicio de sesión sin almacenamiento previo. Esta comprobación funcional no constituye una auditoría completa de seguridad ni una inspección de la base de datos.

Tabla 2. Alcance implementado y evidencia

N.º	Funcionalidad	Estado	Evidencia o ruta
1	Registro, acceso y cierre de sesión	Validada en Render	Auth.jsx; backend/routes/auth.js; PR-02 a PR-06 y PR-18.
2	Alta de ingresos y gastos, búsqueda y eliminación confirmada	Validada en Render	FinanceViews.jsx; #registrar y #historial; PR-07, PR-08, PR-11 y PR-17.
3	Dashboard, resumen mensual, analítica y gráficos	Parcial por ajuste visual	FinanceViews.jsx y charts.js; figuras 2 a 7; pruebas de cálculos y filtros.
4	Tema claro/oscuro y preferencias de moneda	Completa con tasas fijas	usePreferences.js y api.js; PR-05 y PR-12; monedas USD, EUR, PEN y MXN.
5	Temporizador, título y tamaño de ventana sincronizados	Completa	Timer.jsx, useViewport.js y useDocumentTitle.js; PR-16 y pruebas unitarias.
6	Directorio de referencia externo con carga, error y búsqueda	Completa	UserList.jsx; #directorio; PR-15 y PR-16.
7	Interfaz responsive y accesibilidad	Parcial	Tres tamaños sin desbordamiento global; pendientes de contraste y semántica en Lighthouse.

2.2 Funcionalidades planificadas para semanas posteriores

Se propone incorporar edición de movimientos, presupuestos por categoría, exportación de reportes y recuperación de contraseña. Antes de ampliar el alcance se corregirán los hallazgos de accesibilidad y rendimiento y se ampliarán los casos de autorización y recuperación en un entorno de pruebas aislado. El directorio externo seguirá separado de las cuentas reales; los tipos de cambio actuales son valores configurados, no cotizaciones en tiempo real. Las prioridades y semanas propuestas se detallan en el apartado 9.

3. Diseño y arquitectura FrontEnd

3.1 Estructura de componentes

El punto de entrada main.jsx monta App dentro de StrictMode. App coordina la sesión, la vista seleccionada, las preferencias y los mensajes. La navegación usa fragmentos de URL, como #dashboard, dentro de una única aplicación servida desde la raíz.

Jerarquía: App → Auth → AuthLayout para el acceso. Con sesión: App → AppLayout → FinanceViews → Dashboard, Registrar, History, Analytics, Summary o ChartsLayout. Las vistas utilizan layouts y el componente Card. Directorio: App → DirectoryView → UserList y Timer; también se permite acceso público.

Los hooks useFinanceSession, usePreferences, useViewport y useDocumentTitle concentran lógica reutilizable. api.js comunica el frontend con /api/auth y /api/transactions. Express sirve la API y el build; db.js admite PostgreSQL o MySQL según la configuración. La validación funcional utiliza la API desplegada y comprueba que devuelve los datos después de un nuevo inicio de sesión. Las pruebas locales complementarias emplean servicios simulados.

3.2 Uso de React

Tabla 3. Aplicación de los conceptos de React

Concepto	Cómo se utiliza en el proyecto	Componente o archivo de evidencia
Componentes	Piezas funcionales para acceso, vistas financieras, tarjetas, directorio y temporizador.	App.jsx; views/; components/Card.jsx
Props	Card recibe titulo, descripcion, imagen y children. FinanceViews recibe rows, currency, theme y funciones de actualización.	Card.jsx; FinanceViews.jsx
Estado	useState controla pestaña de acceso, tema, filtros, selección, visibilidad y segundos. Los setters actualizan la interfaz.	Auth.jsx; usePreferences.js; Timer.jsx
Eventos e interacción	onClick cambia tema y vistas; onChange actualiza formularios y búsquedas; onSubmit registra operaciones.	Auth.jsx; FinanceViews.jsx
Efectos	Timer y resize usan []; el título depende de title y enabled; UserList consulta al montar y cancela al desmontar.	Timer.jsx; UserList.jsx; useDocumentTitle.js; useViewport.js
Composición	AppLayout contiene la vista activa. Card permite contenido variable mediante children y footer sin duplicar la estructura.	AppLayout.jsx; Card.jsx
Referencias	useRef conserva controladores de peticiones y el identificador del aviso temporal entre renderizados sin dibujar por sí mismo.	useFinanceSession.js; App.jsx

3.3 Organización del código

sidan-frontend/src/views agrupa pantallas y layouts; components contiene elementos reutilizables; hooks concentra sincronización y preferencias. finance.js separa cálculos y filtros, api.js gestiona solicitudes y charts.js administra Chart.js. styles.css conserva el diseño original y App.css sus ajustes.

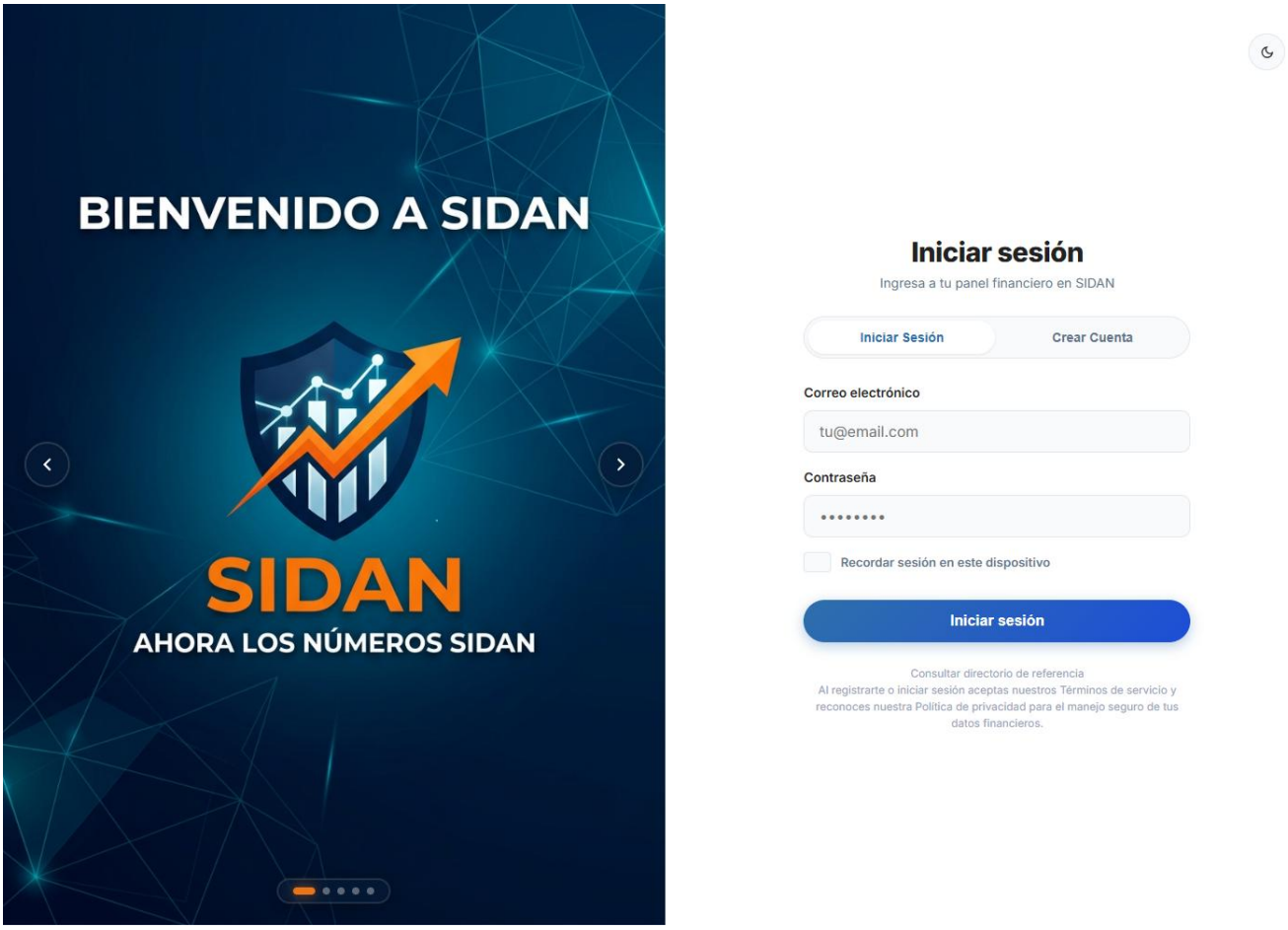
backend contiene servidor, rutas y middleware; tests reúne regresiones; scripts incluye el servidor local de comprobación; docs conserva explicaciones y evidencias. Los componentes usan PascalCase, los hooks el prefijo use y las funciones nombres descriptivos. FinanceViews se carga con lazy y Suspense. Los gráficos se destruyen al limpiar el efecto para no conservar instancias obsoletas.

4. Interfaz, responsive y accesibilidad

4.1 Vistas principales

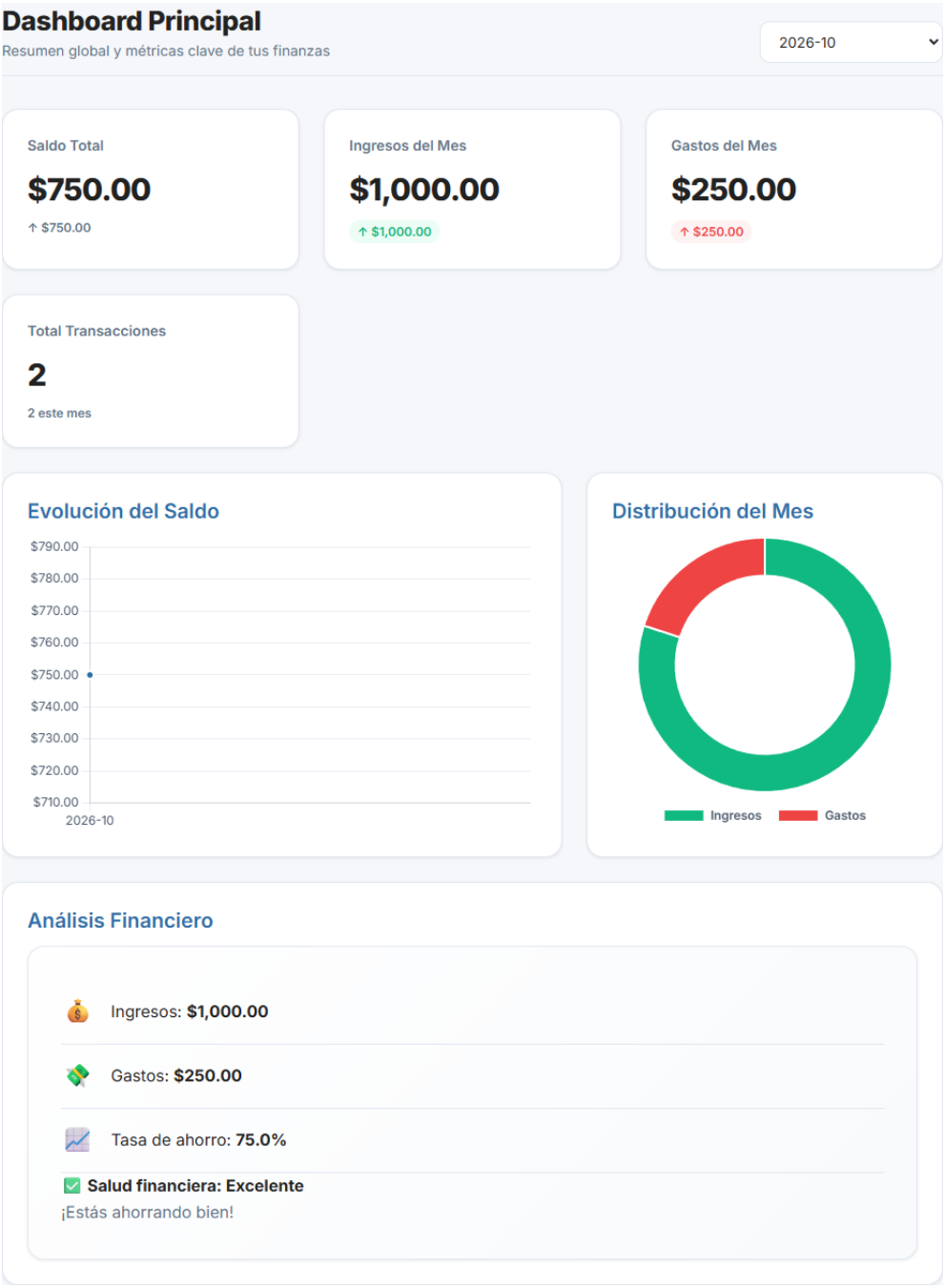
Las figuras 1 a 13 muestran la aplicación alojada en Render durante la validación del 18/10/2026. Se utilizaron una cuenta exclusiva y movimientos identificados como S8-RENDER. Los importes son datos de prueba persistidos en el servidor; no corresponden a finanzas personales reales. Las capturas mantienen la interfaz observada.

Figura 1. Acceso a la aplicación desplegada



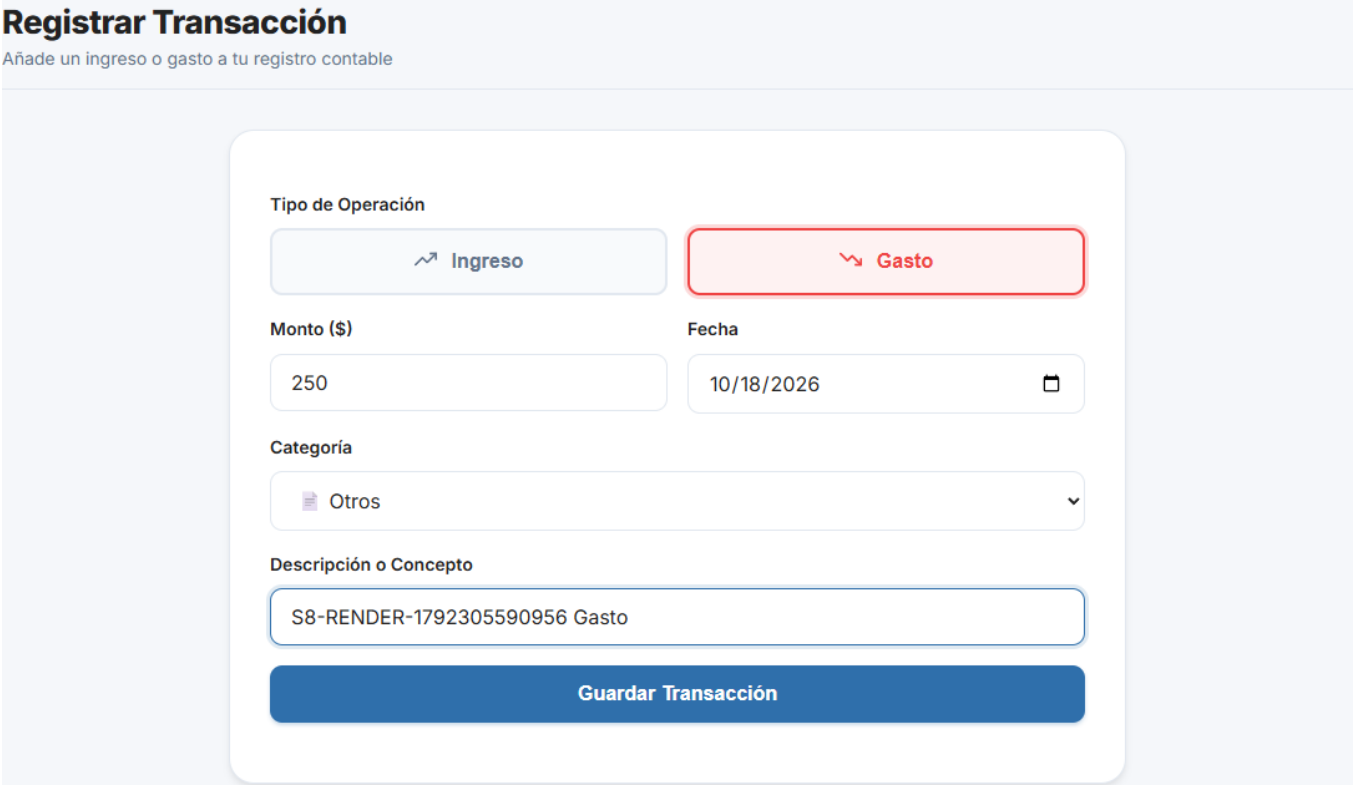
URL pública verificada: https://finanzasmainupgraded.onrender.com/. Se muestran el acceso, el cambio hacia registro y el selector de tema. Se creó una cuenta real exclusiva para esta validación.

Figura 2. Dashboard financiero



Render, cuenta de prueba. Ingreso de 1000 USD y gasto de 250 USD: saldo de 750 USD, dos operaciones y ahorro de 75 %. PR-09 y PR-10 verifican cálculo y persistencia. Las tarjetas reutilizan Card.

Figura 3. Registro de una transacción



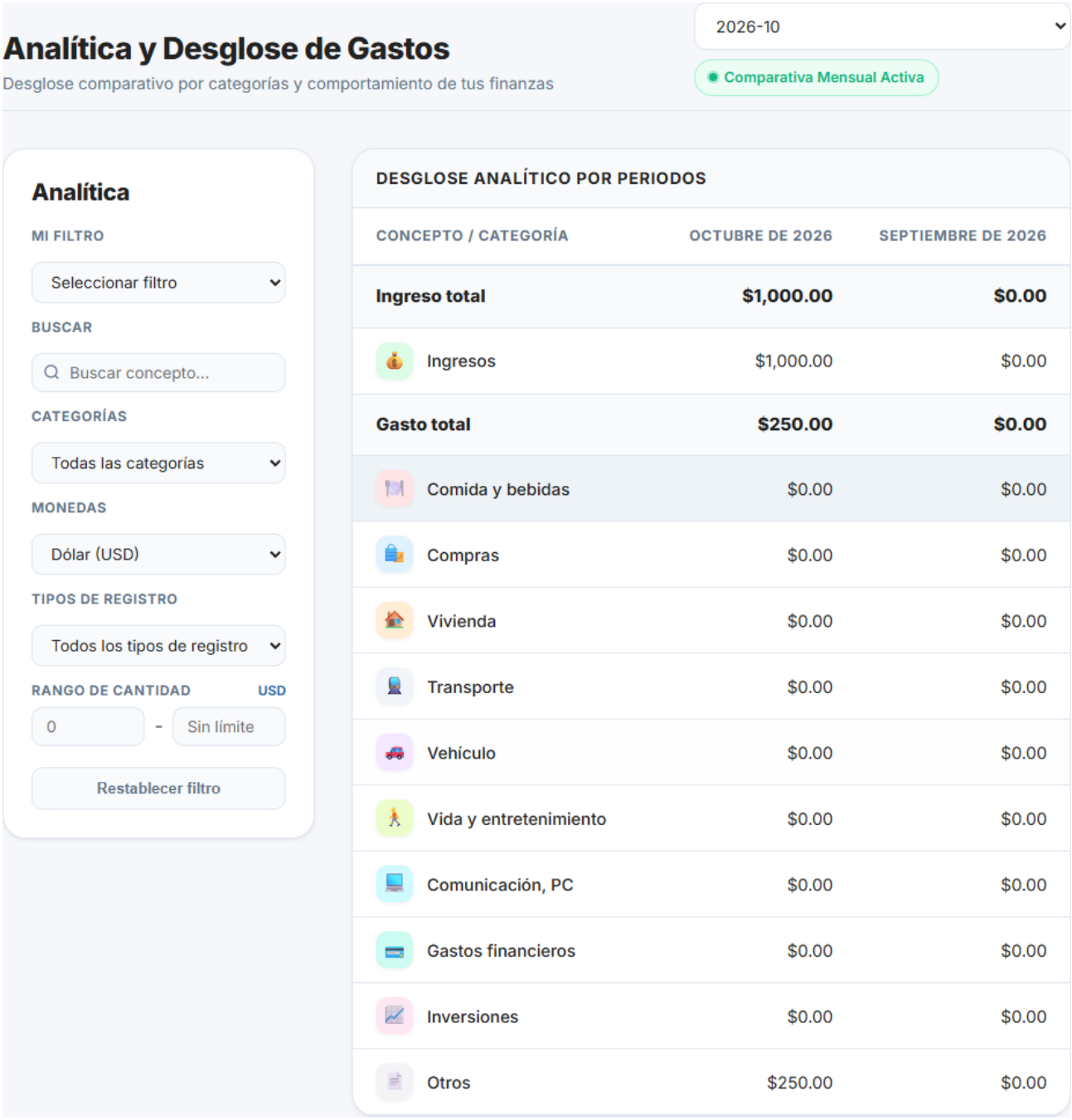
Render. Formulario antes de enviar el gasto de 250 USD con fecha 18/10/2026 y etiqueta de prueba. PR-08 obtuvo HTTP 201 y el identificador 12 de la API real.

Figura 4. Historial de operaciones



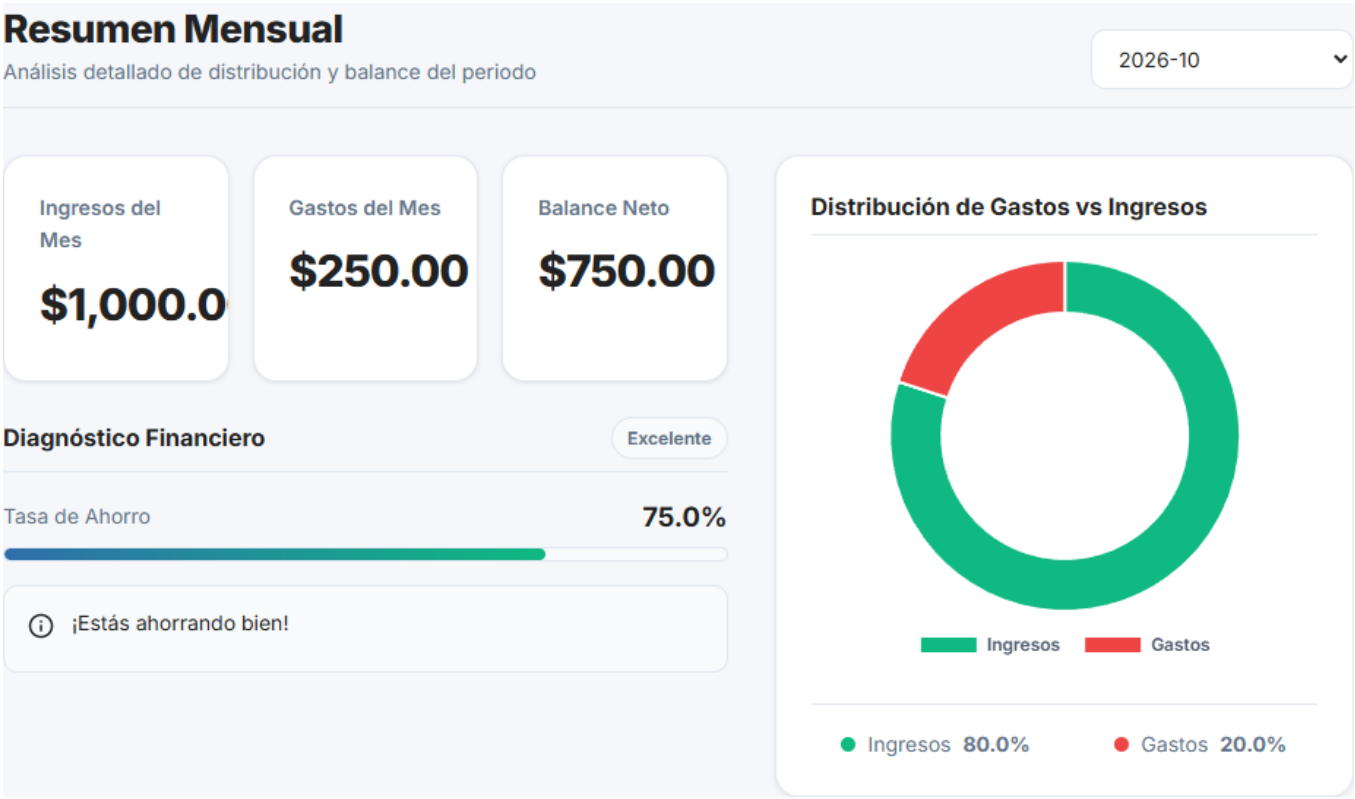
Render. Movimientos 11 y 12 recuperados del servidor después de iniciar sesión desde un nuevo contexto de navegador. PR-11 encontró únicamente el gasto de 250 USD al combinar etiqueta y tipo Gasto.

Figura 5. Analítica por categoría y periodo



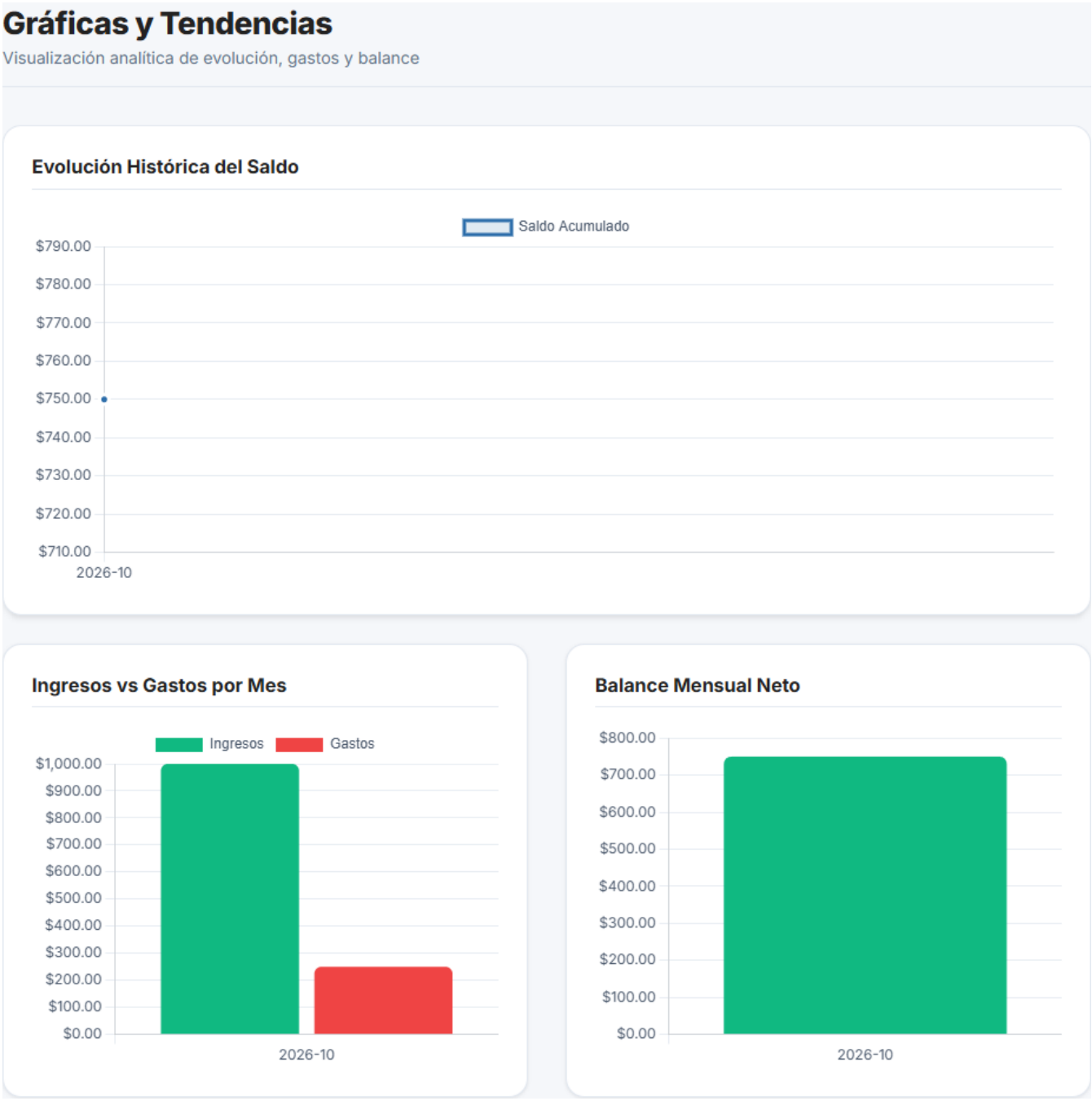
Render. Analítica sobre los dos movimientos persistidos. Presenta comparación entre meses y filtros; los importes se expresan en la moneda seleccionada.

Figura 6. Resumen mensual



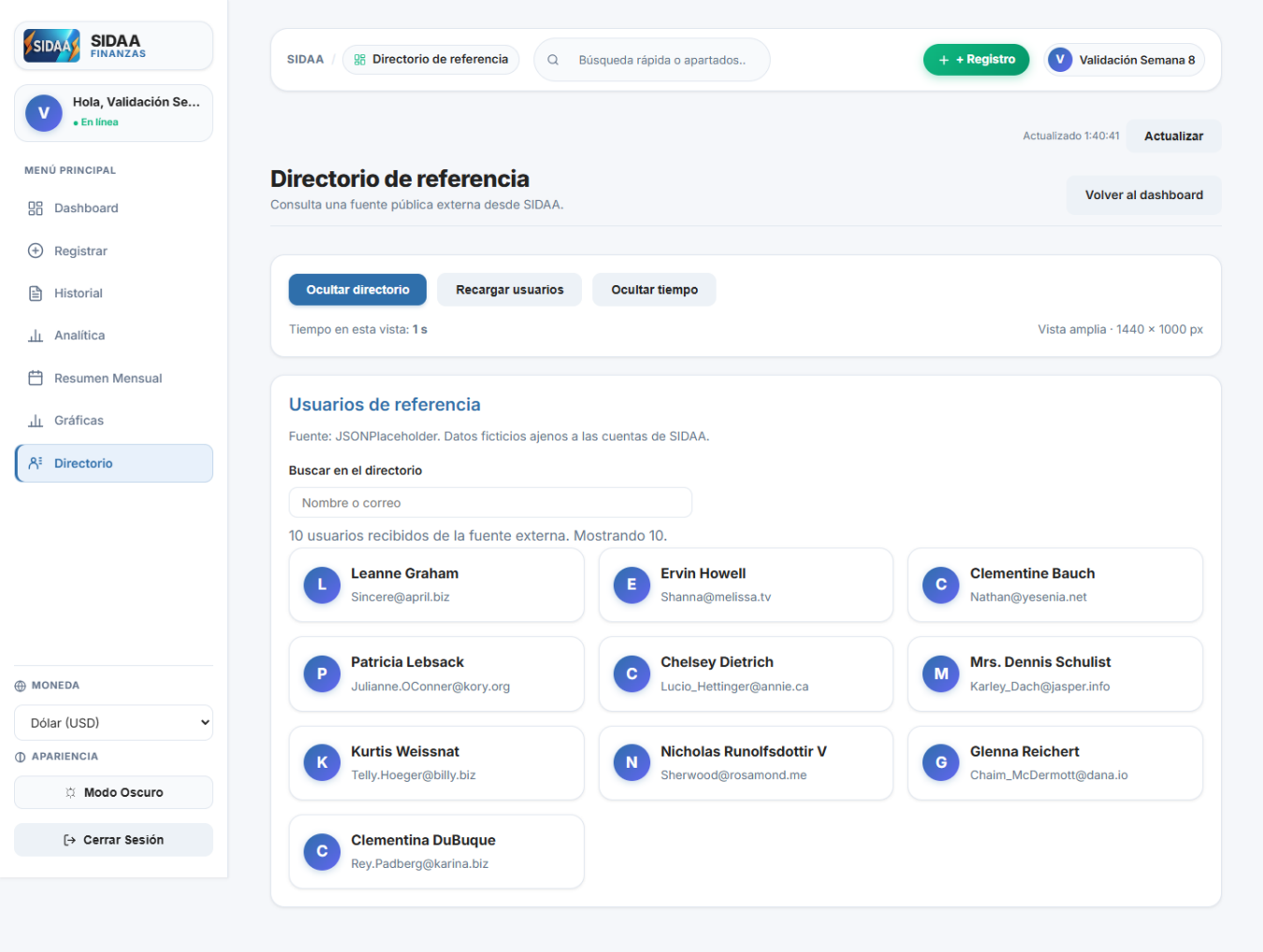
Render a 1280 × 900 px. Resume ingresos, gastos, balance y tasa de ahorro. Se observa recorte del importe de ingresos en su tarjeta; es un defecto visual pendiente, no un cambio del valor calculado.

Figura 7. Gráficas y tendencias



Render. Chart.js representa ingresos, gastos y saldo de la cuenta de prueba. Los movimientos utilizados fueron registrados mediante la API real.

Figura 8. Directorio externo en producción



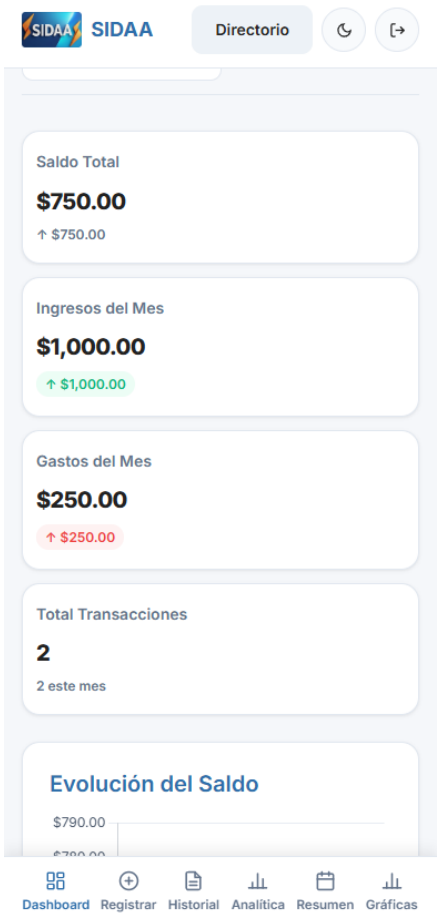
Petición real a JSONPlaceholder: se recibieron diez usuarios ficticios. Se visualizan el temporizador, controles de montaje y dimensiones de ventana. Estos usuarios no son cuentas de SIDAA.

4.2 Diseño adaptable

Tabla 4. Validación del diseño adaptable

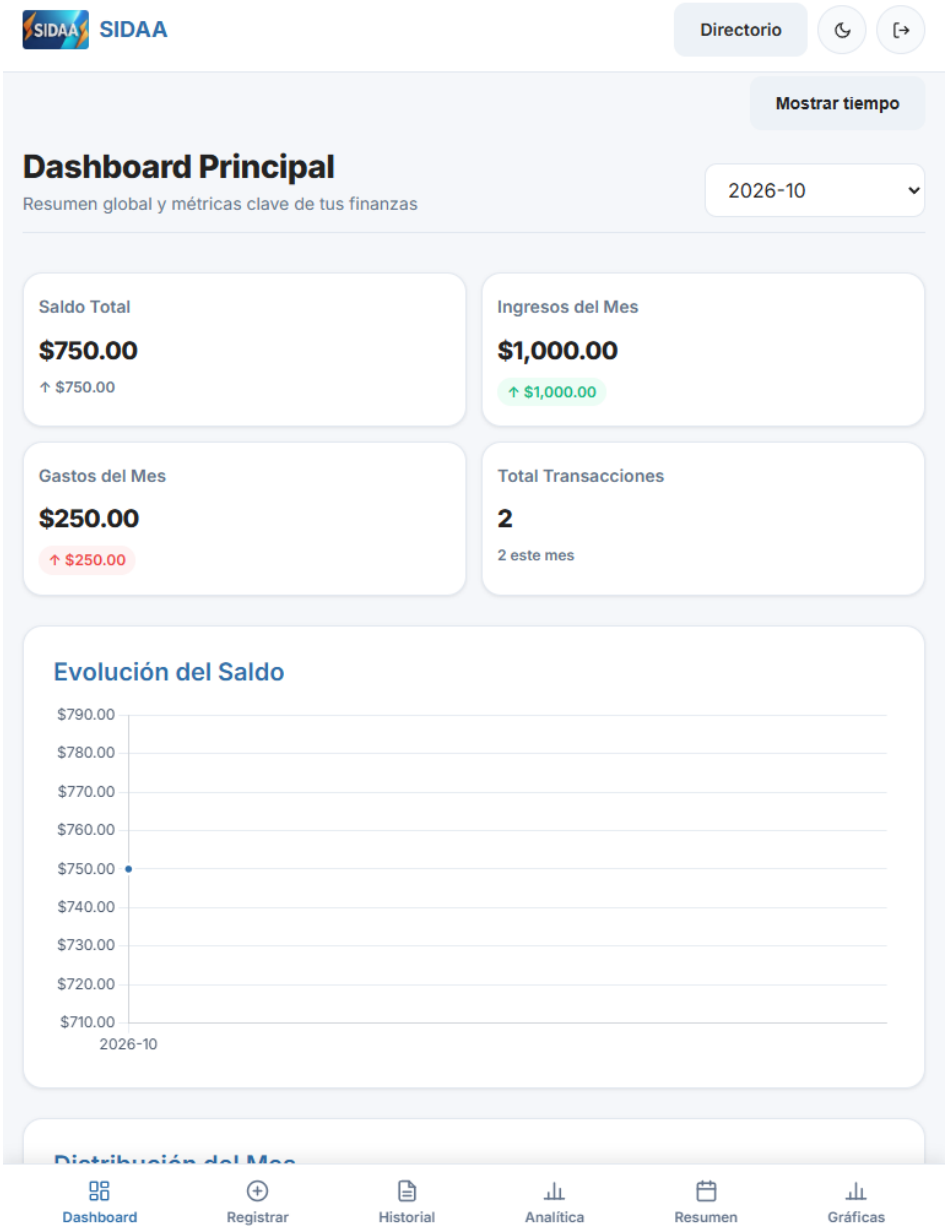
Dispositivo y tamaño probado	Resultado observado	Ajustes realizados
Móvil 390 × 844 px	Dashboard, historial y registro: scrollWidth 390 px. Sin desbordamiento horizontal global; tarjetas apiladas y navegación inferior.	En Render se verificaron las reglas existentes y layout-compact. No se modificó el CSS durante la elaboración del informe.
Tableta 768 × 1024 px	Las tres vistas presentan scrollWidth 768 px. Contenido reorganizado y desplazamiento vertical; layout-wide en el umbral de 768 px.	Se verificó el comportamiento existente en el punto de cambio de diseño.
Escritorio 1440 × 1000 px	Las tres vistas presentan scrollWidth 1440 px. Navegación lateral, métricas distribuidas y espacio para gráficos.	Emulación de viewport. En una captura adicional del resumen a 1280 × 900 px se observó recorte del importe de ingresos; corregir ancho o tipografía de la tarjeta.

Figura 9. Dashboard en viewport móvil



Render. Captura del área visible a 390 × 844 px. El contenido restante se consulta mediante desplazamiento vertical.

Figura 10. Dashboard en viewport de tableta



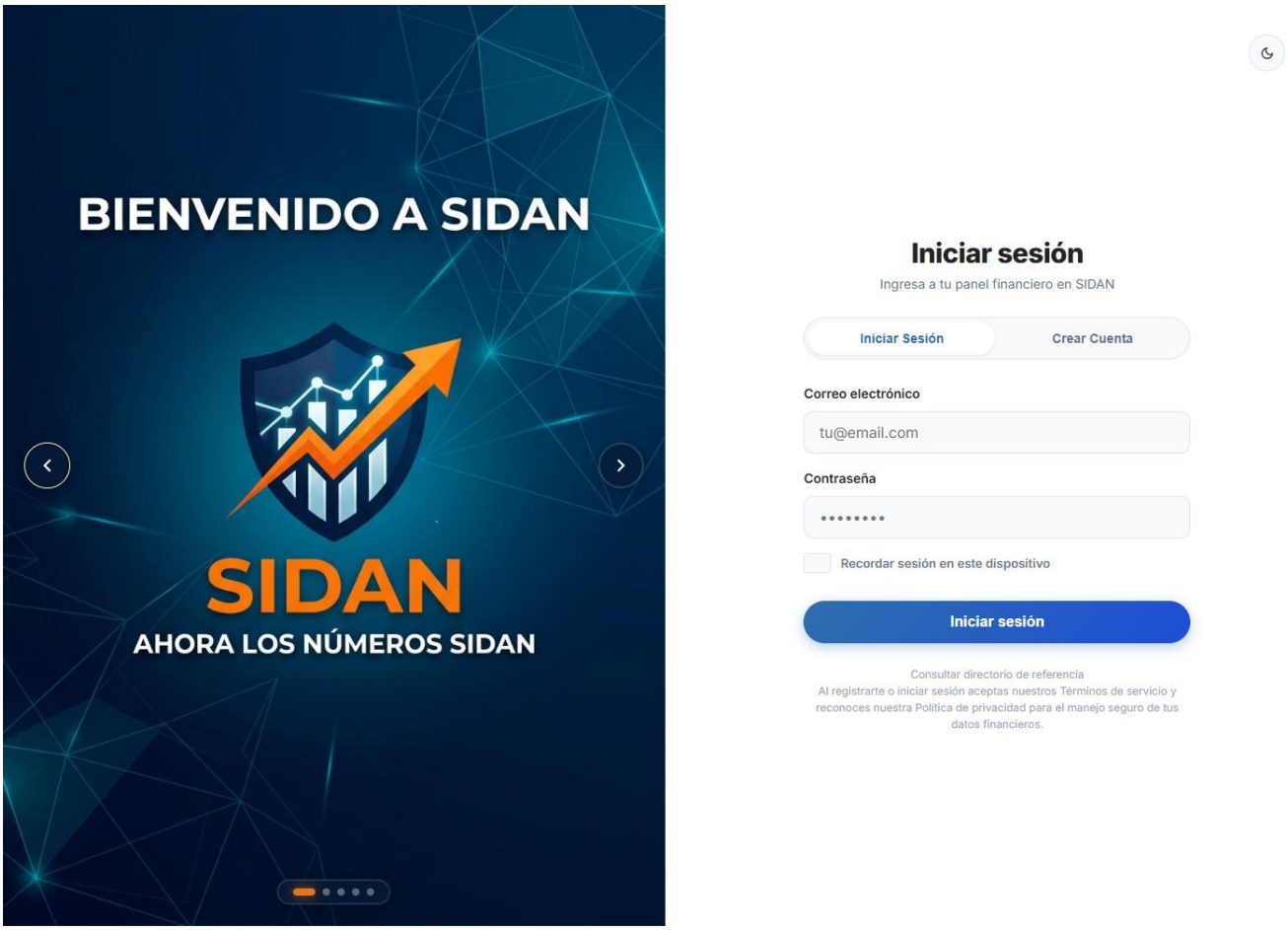
Captura del área visible a 768 × 1024 px. Render, navegador de pruebas con viewport emulado; no es una prueba en una tableta física.

4.3 Accesibilidad

Tabla 5. Accesibilidad observada y acciones pendientes

Verificación	Resultado y evidencia	Mejora pendiente
Etiquetas y estructura semántica	Lighthouse aprobó la asociación de etiquetas; detectó ausencia de landmark main en el acceso. En código existen label, form, section y estados con role=status/alert.	Sustituir el contenedor principal de acceso por main y revisar la jerarquía de encabezados.
Contraste y legibilidad	El aviso legal y enlace al directorio en tema oscuro alcanzan 3,72:1; Lighthouse esperaba 4,5:1 para ese texto. Accesibilidad: 93/100.	Ajustar color del texto sobre #111827 y volver a medir ambos temas.
Teclado y foco visible	Tab alcanzó el botón Slide anterior (carousel-prev); se observó outline blanco automático de 3 px. Figura 11.	Ampliar el recorrido a todos los formularios, filtros y navegación; realizar prueba con lector de pantalla.
Texto alternativo	Lighthouse aprobó image-alt en el acceso. El código contiene alt para la identidad visual y las imágenes del carrusel.	Validar calidad descriptiva en todas las vistas y alternativa textual para gráficos.

Figura 11. Foco de teclado en el acceso



Prueba puntual con Tab en producción. El borde del control anterior del carrusel evidencia la posición del foco; no acredita por sí solo toda la navegación accesible.

5. Pruebas y validación en laboratorio

Pruebas en Render realizadas el 18/10/2026 entre 01:39:57 y 01:41:04 (America/Bogota), con Chrome 155.0.8059.82 y Playwright. Los 18 casos PR siguientes usan HTTPS y la API real, sin interceptar respuestas. Se creó una cuenta exclusiva, se registraron dos movimientos y se eliminaron ambos al terminar. La cuenta queda vacía. Como complemento, node:test aprobó 15 de 15 pruebas locales; Vite y Oxlint terminaron con código 0. Entorno local: Windows, Node 24.19.0 y npm 11.17.0.

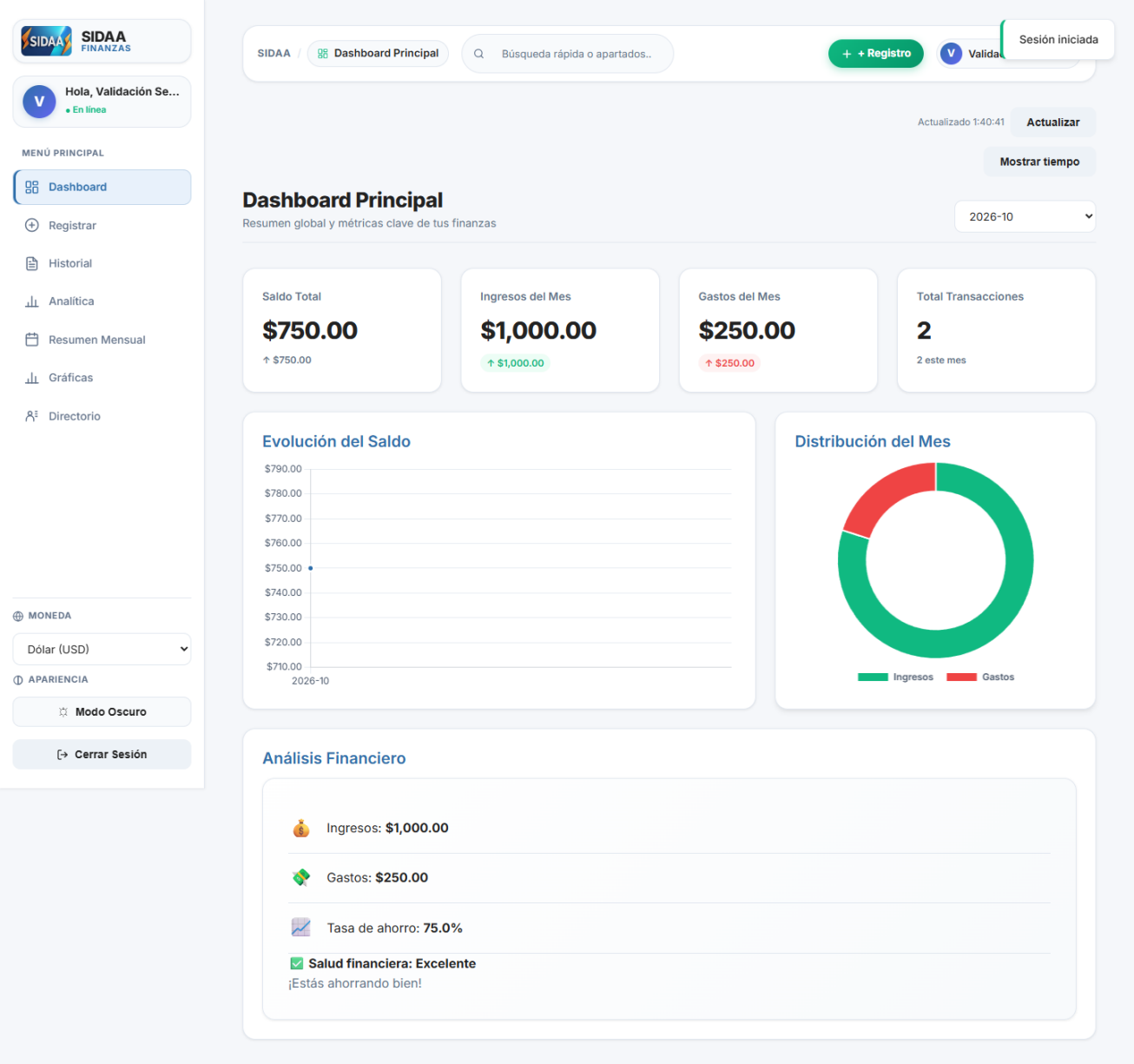
La limpieza de intervalos, eventos y peticiones se verifica con instrumentación en las pruebas automatizadas. La inspección visual del DOM no demuestra por sí sola que todos los recursos hayan sido liberados.

Tabla 6. Casos funcionales ejecutados en Render

Caso de prueba	Procedimiento	Resultado esperado	Resultado obtenido
PR-01	Abrir la raíz pública y consultar /api/health.	HTTP 200 y entorno de producción.	Aprobado. Raíz 200; health 200; status=ok; environment=production.
PR-02	Seleccionar Crear Cuenta.	Solo se monta el formulario elegido.	Aprobado. Registro visible; cero formularios de acceso.
PR-03	Enviar nombre, correo exclusivo y contraseña por el formulario.	Crear la cuenta en el servidor y volver al acceso.	Aprobado. POST /api/auth/register: 201. Se reutilizó esta cuenta durante todo el recorrido.
PR-04	Enviar una contraseña incorrecta para la cuenta creada.	Rechazar el acceso sin mostrar finanzas.	Aprobado. POST /api/auth/login: 401; formulario visible; dashboard ausente.
PR-05	Cambiar el tema y recargar la página.	Conservar la preferencia.	Aprobado. light → dark; recarga mantiene dark.
PR-06	Iniciar sesión con la contraseña correcta.	Acceder a una cuenta sin movimientos.	Aprobado. Login 200; GET /api/transactions: 200 y arreglo vacío.
PR-07	Crear ingreso de 1000 USD, fecha 18/10/2026.	Guardar y reflejar la operación.	Aprobado. POST /api/transactions: 201; id 11.
PR-08	Crear gasto de 250 USD, fecha 18/10/2026.	Guardar y actualizar los valores.	Aprobado. POST /api/transactions: 201; id 12.
PR-09	Consultar el dashboard después de ambas altas.	Ingreso 1000; gasto 250; saldo 750; dos operaciones.	Aprobado. Coinciden los cuatro indicadores mostrados.
PR-10	Recargar; abrir otro contexto sin cookies ni almacenamiento e iniciar sesión.	Recuperar los mismos dos registros del servidor.	Aprobado. GET 200; ids 11 y 12; fecha e importes conservados; saldo $750.00.
PR-11	Buscar la etiqueta S8-RENDER y seleccionar Gasto.	Obtener una sola coincidencia.	Aprobado. Una fila, 18/10/2026, gasto $250.00.
PR-12	Cambiar la moneda de USD a PEN y volver a USD.	Aplicar factor configurado 3,72.	Aprobado. Saldo S/2,790.00. Tasa fija de referencia; no cotización en vivo.
PR-13	Abrir el resumen mensual.	Ingresos 1000; gastos 250; balance 750; ahorro 75 %.	Aprobado. Valores correctos; defecto visual de recorte registrado aparte.

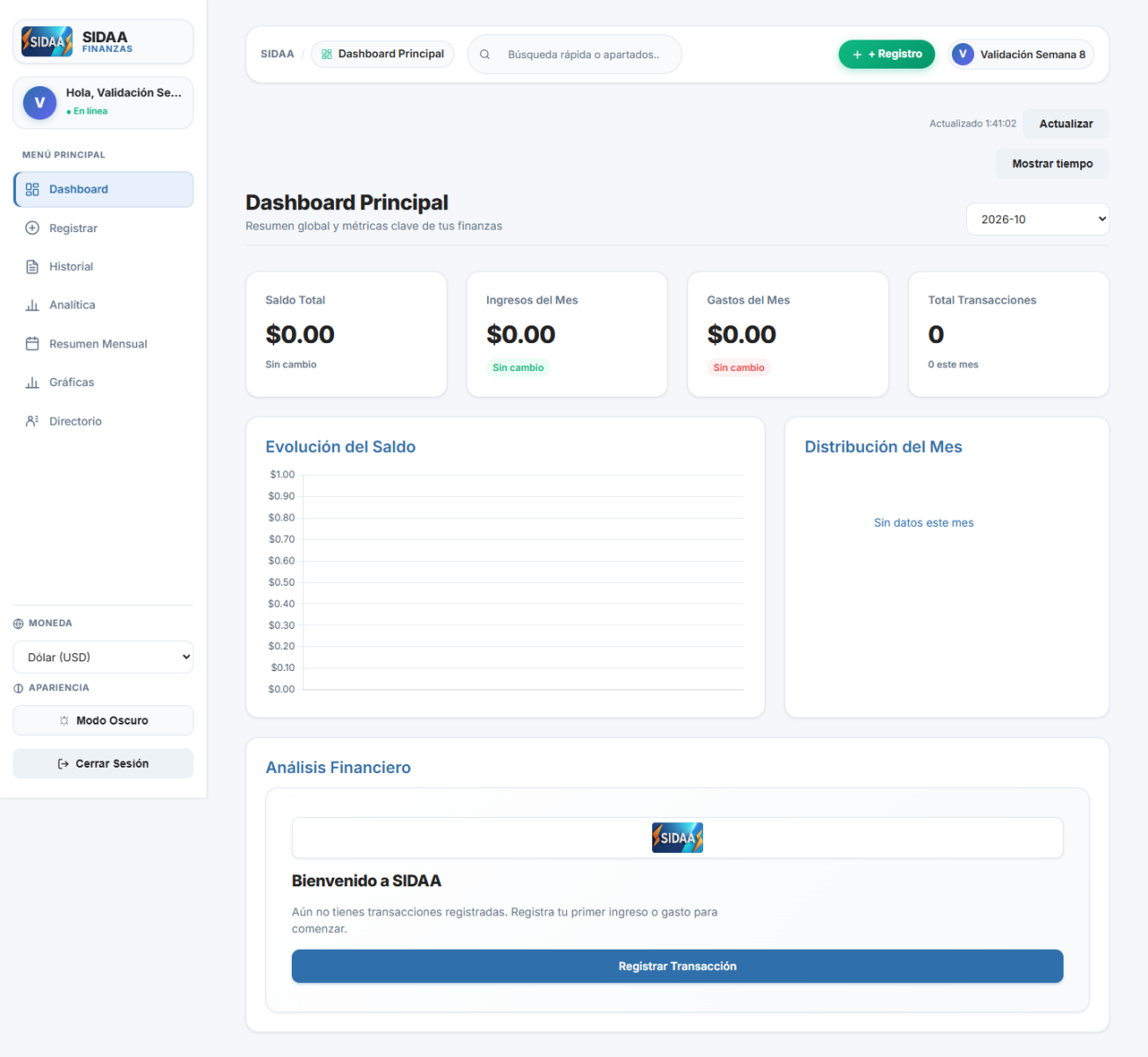
Caso de prueba	Procedimiento	Resultado esperado	Resultado obtenido
PR-14	Abrir dashboard, historial y registro en tres tamaños.	No superar el ancho de cada viewport.	Aprobado. Nueve combinaciones sin desbordamiento global: 390, 768 y 1440 px.
PR-15	Abrir directorio y buscar Leanne.	Cargar diez usuarios y filtrar uno.	Aprobado. Diez usuarios ficticios recibidos; una coincidencia.
PR-16	Ocultar y mostrar Timer; ocultar directorio.	Retirar componentes y reiniciar el contador.	Aprobado. Cero timers al ocultar; 0 s al mostrar; cero listas al ocultar.
PR-17	Seleccionar solo los movimientos creados, confirmar borrado y recargar.	Eliminar ambos registros de forma persistente.	Aprobado. DELETE ids 12 y 11: 200; GET posterior: 200 y []; dashboard vacío.
PR-18	Cerrar sesión y consultar movimientos sin token.	Volver al acceso y denegar consulta.	Aprobado. Acceso visible; dashboard ausente; GET sin token: 401.

Figura 12. Persistencia comprobada en Render



PR-10. Dashboard después de iniciar sesión desde un contexto nuevo de navegador, sin almacenamiento previo. La API devolvió los ids 11 y 12 con importes 1000 y 250 USD; saldo visible de 750 USD.

Figura 13. Limpieza final de los movimientos de prueba



PR-17. Tras DELETE 200 de los ids 12 y 11 y una nueva consulta GET 200 con [], el dashboard queda en cero. La cuenta de prueba permanece vacía; la API no dispone de eliminación de cuentas.

6. Auditoría con Lighthouse

URL auditada: https://finanzasmainupgraded.onrender.com/

Fecha de auditoría: 18/10/2026, 01:14 (America/Bogota).

Entorno: Lighthouse CLI 13.5.0; Chrome 155; acceso público sin sesión, tema oscuro, móvil emulado de 412 × 823 px, escala 1,75. Red simulada: RTT 150 ms y 1638,4 Kbps; CPU ×4. Las puntuaciones corresponden a una ejecución y pueden variar.

Tabla 7. Resultados de Lighthouse en producción

Categoría	Puntuación	Hallazgo relevante	Acción o mejora
Rendimiento	76/100	FCP 2,8 s; LCP 4,6 s; Speed Index 4,7 s; TBT 0 ms; CLS 0.	Optimizar imágenes y recursos que bloquean el renderizado; repetir medición.
Accesibilidad	93/100	Contraste 3,72:1 en aviso/enlace; falta landmark main.	Ajustar contraste y contenedor semántico del acceso.
Buenas prácticas	100/100	Sin fallos puntuables en esta categoría para la página auditada.	Mantener comprobación por versión; no equivale a auditoría de seguridad del backend.
SEO	100/100	Cumple las comprobaciones básicas aplicables al acceso público.	Revisar metadatos al incorporar nuevas páginas públicas; no implica posicionamiento garantizado.

Hallazgo 1. El aviso legal y el enlace al directorio usan #64748b sobre #111827. Lighthouse calcula una relación 3,72:1, menor que 4,5:1 para el texto evaluado. La acción propuesta es aumentar el contraste y verificarlo en ambos temas; esta corrección aún no se aplicó.

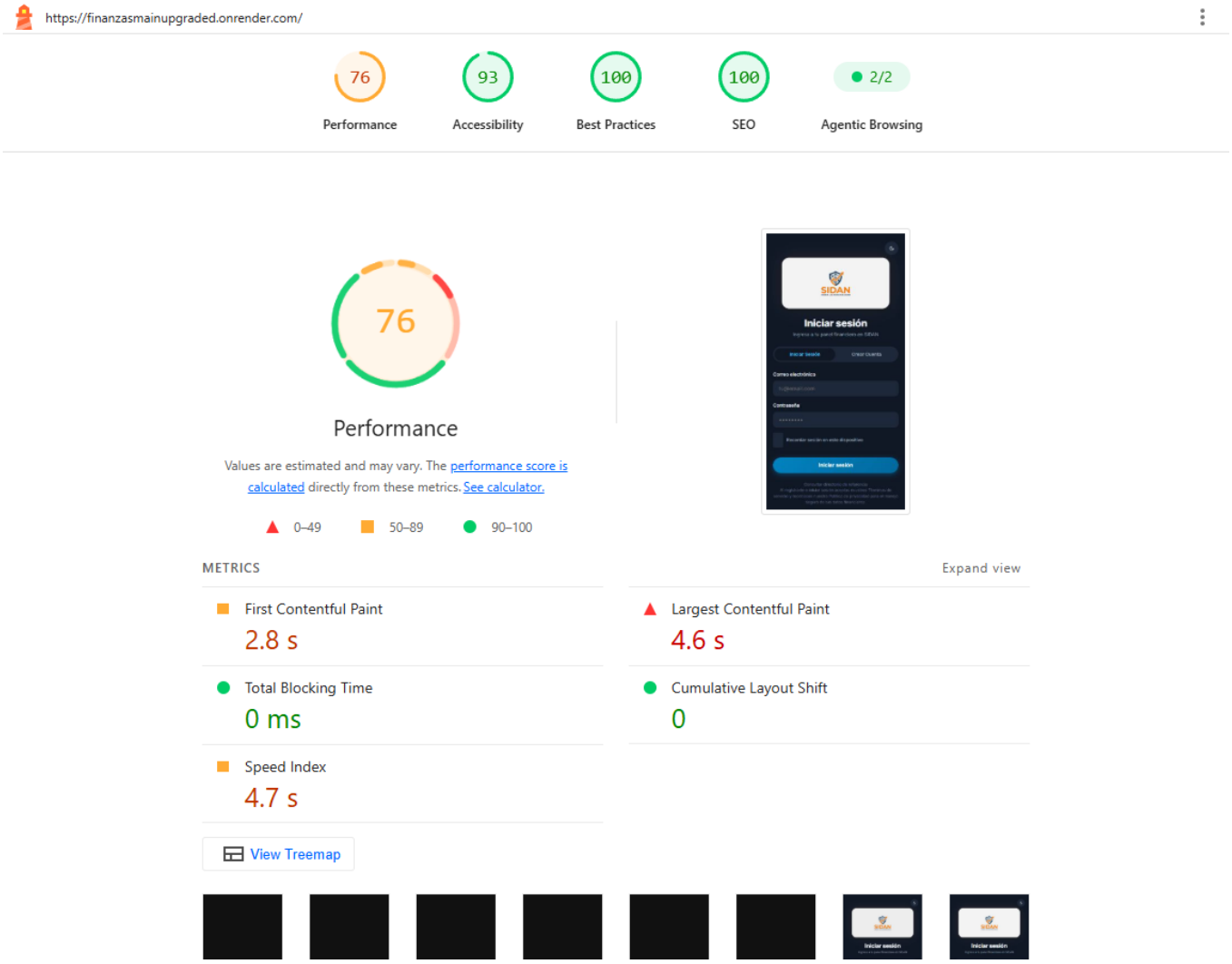
Hallazgo 2. La imagen móvil sidan-logo.png pesa 182 289 bytes y mide 1024 × 571 píxeles. Lighthouse estima un ahorro de 169 109 bytes (165 KiB) mediante formato moderno, compresión y tamaño adecuado. Se propone WebP/AVIF y dimensiones explícitas; el ahorro es una estimación de la herramienta, no un resultado ya conseguido.

Hallazgo 3. CSS y la hoja de Google Fonts bloquean el renderizado; la herramienta estima 1510 ms de ahorro potencial. Se propone revisar la carga de fuentes y estilos críticos. El LCP de 4,6 s justifica priorizar esta optimización.

Incidencia de disponibilidad. El primer intento de auditoría respondió HTTP 503 y no produjo puntuaciones válidas. Una comprobación posterior devolvió HTTP 200 en la raíz y en /api/health; el segundo intento público produjo los valores de la tabla 7. No se determinó la causa del 503. También se auditó el entorno local: 62/93/100/100; esas cifras no se sustituyen por las de producción.

Alcance. Lighthouse evaluó la pantalla de acceso. Las vistas autenticadas se comprobaron funcionalmente en Render en los casos PR-06 a PR-14 y PR-17. La auditoría automática no sustituye pruebas con usuarios, lector de pantalla o una auditoría de seguridad.

Figura 14. Reporte Lighthouse de la URL pública



Captura del reporte generado: 76 de rendimiento, 93 de accesibilidad, 100 de buenas prácticas y 100 de SEO. El reporte HTML y JSON íntegro se incluye como adjunto del PDF y en docs/examen/evidencias del repositorio; la captura y los hallazgos permanecen dentro del informe.

7. Repositorio y despliegue

Tabla 8. Repositorio despliegue y herramientas

Recurso	Enlace o información
Repositorio Git	https://github.com/ramko09-oss/finanzasmainupgraded Acceso público HTTP 200; consulta remota de main satisfactoria.
Aplicación desplegada	https://finanzasmainupgraded.onrender.com/ Directorio público: /#directorio. Raíz y /api/health: HTTP 200; flujo financiero y persistencia comprobados en la URL pública.
Rama y versión entregada	Rama main. Código evaluado: f736e2eaf5acad9194d0135eb8b2261534ee3139. Los commits documentales posteriores organizan la entrega sin modificar la aplicación evaluada.
Ejecución local	Node 20.19+ (rama 20) o 22.12+; npm; base de datos configurada. Ver secuencia reproducible debajo de la tabla.
Dependencias principales	React/React DOM 19.2.8, Vite 8.3.4 usado en la compilación, Chart.js 4.5.1, Express, pg/mysql2, bcryptjs y jsonwebtoken. Pruebas: node:test, JSDOM y Oxlint.

Ejecución local con base de datos

La guía completa se incorpora en el anexo 5 y se entrega también como Word independiente. Requiere Git, Node.js 22.12+ de la rama 22 o Node.js 24, npm y PostgreSQL de pruebas accesible con SSL. El archivo .env se crea en la raíz del repositorio con valores propios.

Desde la raíz: npm run build; para una base nueva, node backend/init_postgres_db.js; finalmente npm start y abrir http://localhost:3001. El inicializador crea tablas en una base ya existente. Para desarrollo, mantener el backend activo y ejecutar npm run dev:react en otra terminal.

Validación reproducible sin base de datos

npm run build
npm test
npm --prefix sidan-frontend run lint
npm run preview:integration
Abrir http://127.0.0.1:3002 y usar credenciales de prueba. Este servidor simula la API, no verifica contraseñas ni almacena datos de forma permanente.

La publicación usa .github/workflows/deploy.yml al recibir push en main. GitHub Actions solicita el despliegue mediante el secreto configurado y Auto-Deploy permanece desactivado. El código evaluado es f736e2eaf5acad9194d0135eb8b2261534ee3139; la actualización de la entrega organiza documentos y evidencias sin cambiar ese código.

8. Dificultades y decisiones técnicas

Dificultad 1. Conservar el diseño durante la migración. La separación entre una interfaz anterior y una incorporación React podía generar rutas o pantallas duplicadas. La solución existente centraliza React en la raíz, reutiliza styles.css y los layouts y redirige enlaces antiguos /react/. La prueba de Express confirma ese comportamiento. Aprendizaje: la migración debe preservar un único punto de entrada y componentes visuales reutilizables.

Dificultad 2. Procesos activos después del desmontaje. Intervalos, listeners y respuestas tardías podían seguir asociados a vistas retiradas. Se implementaron clearInterval, removeEventListener y AbortController en las funciones de limpieza. Las pruebas con StrictMode verificaron un único intervalo activo y cero tras desmontar. Aprendizaje: cada efecto debe declarar dependencias y responsabilizarse de liberar sus recursos.

Dificultad 3. Error transitorio frente a sesión inválida. Ambos casos podrían dejar al usuario sin contexto si se trataran igual. La implementación conserva los últimos datos ante un 503 y limpia la sesión ante 401/403. Los casos locales CP-07 y CP-08 del laboratorio anterior reprodujeron ambos escenarios; estos fallos se provocaron únicamente en el simulador. PR-04 comprueba en Render el rechazo real de una contraseña incorrecta. Aprendizaje: distinguir conectividad de autorización permite responder con mensajes y estados adecuados.

Dificultad 4. Primera auditoría pública fallida. El acceso respondió 503 en el primer intento. Se conservó ese resultado, se verificaron nuevamente los enlaces y se repitió la auditoría cuando respondían 200. No se atribuye el fallo a una causa no comprobada. Aprendizaje: registrar intentos fallidos y distinguir métricas válidas de una página de error.

Las acciones correctivas del informe están propuestas; los resultados medidos corresponden al código previo a esas mejoras.

9. Plan de continuidad del proyecto

Tabla 9. Plan de continuidad propuesto

Mejora o funcionalidad pendiente	Prioridad	Semana prevista	Responsable propuesto
Corregir contraste, landmark main, recorrido de teclado y recorte de importes del resumen.	Alta	9	Edward Villanueva y Franco Atencia
Optimizar imagen móvil, fuentes y CSS; repetir Lighthouse.	Alta	9–10	Joseph Velasquez y Edward Villanueva
Ampliar autorización entre cuentas, validaciones negativas y recuperación de datos en una base aislada.	Alta	10	Kevin Ramirez y Franco Atencia
Implementar edición de movimientos y presupuestos por categoría.	Media	11–12	Kevin Ramirez y Diego Gamarra
Exportar reportes e incorporar recuperación de contraseña.	Media	13–14	Kevin Ramirez, Diego Gamarra y Alexis Sanchez
Validación integral, revisión documental y preparación de exposición final.	Alta	15–16	Todo el equipo; coordinación de Franco Atencia

10. Conclusiones

1. El MVP integra las operaciones financieras principales en una sola aplicación React y conserva la identidad visual previa. Su arquitectura separa presentación, lógica financiera, comunicación API y sincronización.

2. Componentes, props y useState permiten reutilización y cambios visuales coherentes. Los efectos con limpieza gestionan temporizadores, eventos, título y solicitudes; su comportamiento se comprobó en pruebas con StrictMode.

3. Los 18 casos de Render se aprobaron. La API real creó la cuenta y dos movimientos, devolvió los mismos registros tras un nuevo inicio de sesión y confirmó su eliminación. Las 15 pruebas locales aportan cobertura adicional de lógica y limpieza de recursos; la validación no acredita seguridad integral ni pruebas con usuarios finales.

4. El resumen mostró un recorte visual del importe a 1280 px, pendiente de corrección. Las nueve combinaciones de vista y tamaño no mostraron desbordamiento horizontal global. Lighthouse obtuvo 76/93/100/100 en el acceso público y detectó mejoras concretas de contraste, semántica y optimización de recursos.

5. La continuidad prioriza los hallazgos documentados antes de ampliar funciones. El video grupal ya está publicado y se enlaza en el apartado 13. Las evidencias distinguen resultados observados, pruebas complementarias y mejoras futuras.

11. Referencias

Meta. (s. f.). Passing Props to a Component. React. https://react.dev/learn/passing-props-to-a-component (consulta: 18/10/2026).

Meta. (s. f.). Synchronizing with Effects. React. https://react.dev/learn/synchronizing-with-effects (consulta: 18/10/2026).

Vite. (s. f.). Getting Started. https://vite.dev/guide/ (consulta: 18/10/2026).

Google. (s. f.). Introduction to Lighthouse. Chrome for Developers. https://developer.chrome.com/docs/lighthouse/overview (consulta: 18/10/2026).

Proyecto SIDAA Finanzas. (2026). Código fuente, README, REACT-VERIFICATION y EFFECTS. Revisión f736e2e. https://github.com/ramko09-oss/finanzasmainupgraded

JSONPlaceholder. (s. f.). Users endpoint. https://jsonplaceholder.typicode.com/users (consulta efectiva desde la aplicación: 18/10/2026). Fuente de usuarios ficticios de referencia.

Recursos gráficos. Logo institucional conservado de la plantilla proporcionada. Imágenes de SIDAA/SIDAN incluidas en sidan-frontend/public/img. Capturas propias de las ejecuciones de validación; no se incorporaron imágenes de bancos externos para este informe.

12. Declaración de participación del equipo

Los aportes siguientes corresponden a la distribución descrita por el equipo al inicio de su video de exposición, entre 00:03 y 00:51. Esta fuente sustituye los roles genéricos del borrador. Se omiten códigos y otros datos de portada no proporcionados.

Tabla 10. Integrantes y aportes descritos en la exposición

Integrante	Responsabilidad	Aporte declarado	Contenido relacionado
Franco Valentino Atencia Cabrera	Coordinación y documentación	Coordinación del equipo, revisión documental y seguimiento de actividades.	Introducción y distribución de aportes.
Edward David Villanueva Martinez	Interfaz y accesibilidad	Interfaz, accesibilidad y pruebas del sistema.	Interfaz, adaptación y accesibilidad.
Joseph Adrian Velasquez Rivas	Arquitectura frontend	Arquitectura y diseño de la interfaz de usuario.	Componentes y organización frontend.
Kevin Isaac Ramirez Alayo	Backend y validaciones	Backend, validaciones y pruebas del sistema.	API, persistencia y despliegue.
Diego Reydhel Gamarra de la Cruz	Funciones financieras	Desarrollo de funciones financieras y gestión de movimientos.	Ingresos, gastos, saldo e historial.
Alexis Sandro Sanchez Ramos	Integración de componentes	Integración de componentes y funcionalidades del sistema.	Integración y consulta externa.

13. Enlaces de entrega y checklist final

Aplicación React desplegada: https://finanzasmainupgraded.onrender.com/
Repositorio: https://github.com/ramko09-oss/finanzasmainupgraded
Rama: main. Commit del código evaluado: f736e2eaf5acad9194d0135eb8b2261534ee3139.

El informe incluye problema, usuario objetivo, alcance, arquitectura, props, estado, eventos, efectos, responsive, accesibilidad, pruebas, métricas, despliegue, aportes individuales y continuidad. Conserva los 18 casos funcionales en Render y las 15 pruebas locales como resultados de sus ejecuciones documentadas.

Las evidencias incluyen capturas del MVP, prueba manual puntual de teclado y foco, auditoría Lighthouse y al menos dos hallazgos con su plan de mejora. El HTML y JSON completos se adjuntan al PDF y están disponibles en el repositorio. Las instrucciones locales se integran en el anexo 5 y se entregan como Word independiente.

Video grupal publicado: https://youtu.be/FHAjr41D-Qc

La exposición incluye la presentación del equipo, capturas del producto, fragmentos de código y resultados de las pruebas. Los apartados 4 a 6 de este informe conservan las evidencias del MVP, la prueba manual de accesibilidad, las métricas y los hallazgos explicados con su plan de mejora.

Anexo 1 Registro de pruebas automatizadas

Salida de npm test. Se omiten únicamente los tiempos individuales; se conserva el nombre y resultado de cada caso. Ejecución del 18/10/2026. Duración total registrada: 19 360,6554 ms.

PA-01 APROBADO. Timer limpia cada intervalo de StrictMode y comienza desde cero al volver a montarse.

PA-02 APROBADO. resize sincroniza el estado y elimina el mismo listener al desmontarse.

PA-03 APROBADO. El título responde a sus dependencias y restaura el documento al desmontarse.

PA-04 APROBADO. UserList muestra carga, datos externos y aborta las peticiones de sus montajes.

PA-05 APROBADO. UserList presenta errores HTTP y no actualiza una vista desmontada con una respuesta tardía.

PA-06 APROBADO. UserList termina una espera excesiva y limpia el plazo al desmontarse.

PA-07 APROBADO. La comparación de enero usa diciembre del año anterior y conserva el saldo acumulado.

PA-08 APROBADO. La búsqueda del historial combina tipo, concepto y fecha sin modificar los registros.

PA-09 APROBADO. Los filtros analíticos comparan importes en la moneda seleccionada e interpretan categorías existentes.

PA-10 APROBADO. Express sirve React desde la raíz y los enlaces antiguos redirigen sin crear otra interfaz.

PA-11 APROBADO. El acceso conserva las sesiones temporales y recordadas sin mezclar almacenamientos.

PA-12 APROBADO. Los nuevos movimientos conservan su fecha y se almacenan en USD desde la moneda elegida.

PA-13 APROBADO. Un token expirado limpia ambas sesiones y permite a React mostrar su acceso sin cambiar la ruta.

PA-14 APROBADO. Un error transitorio conserva la sesión y se propaga con su estado HTTP.

PA-15 APROBADO. Las fechas del primer día permanecen en el mes correcto y los importes usan los cálculos originales.

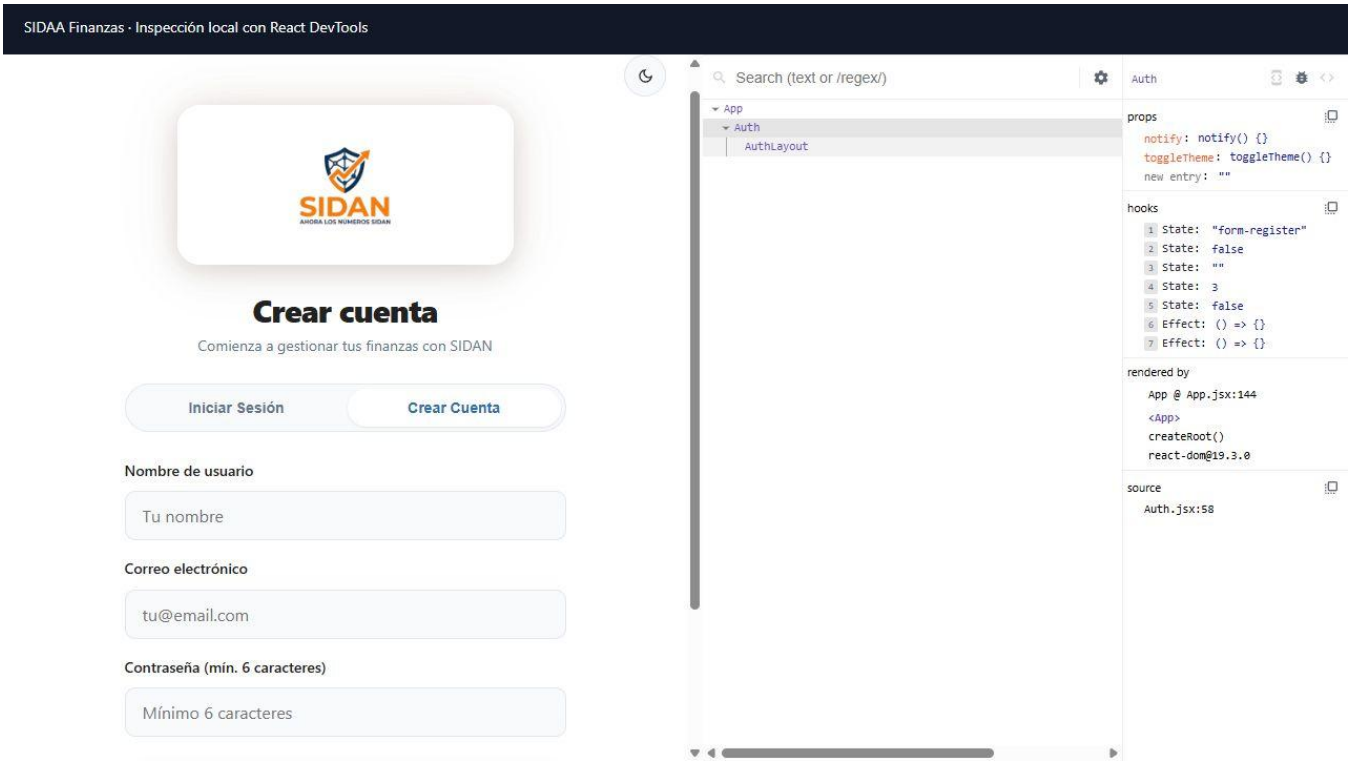
Resumen de ejecución: tests 15 · pass 15 · fail 0 · cancelled 0 · skipped 0 · todo 0.

Compilación: Vite 8.3.4, 43 módulos transformados, finalizada sin error. Análisis estático: Oxlint finalizado sin diagnósticos y con código de salida 0.

Anexo 2 Evidencia técnica de estados y efectos

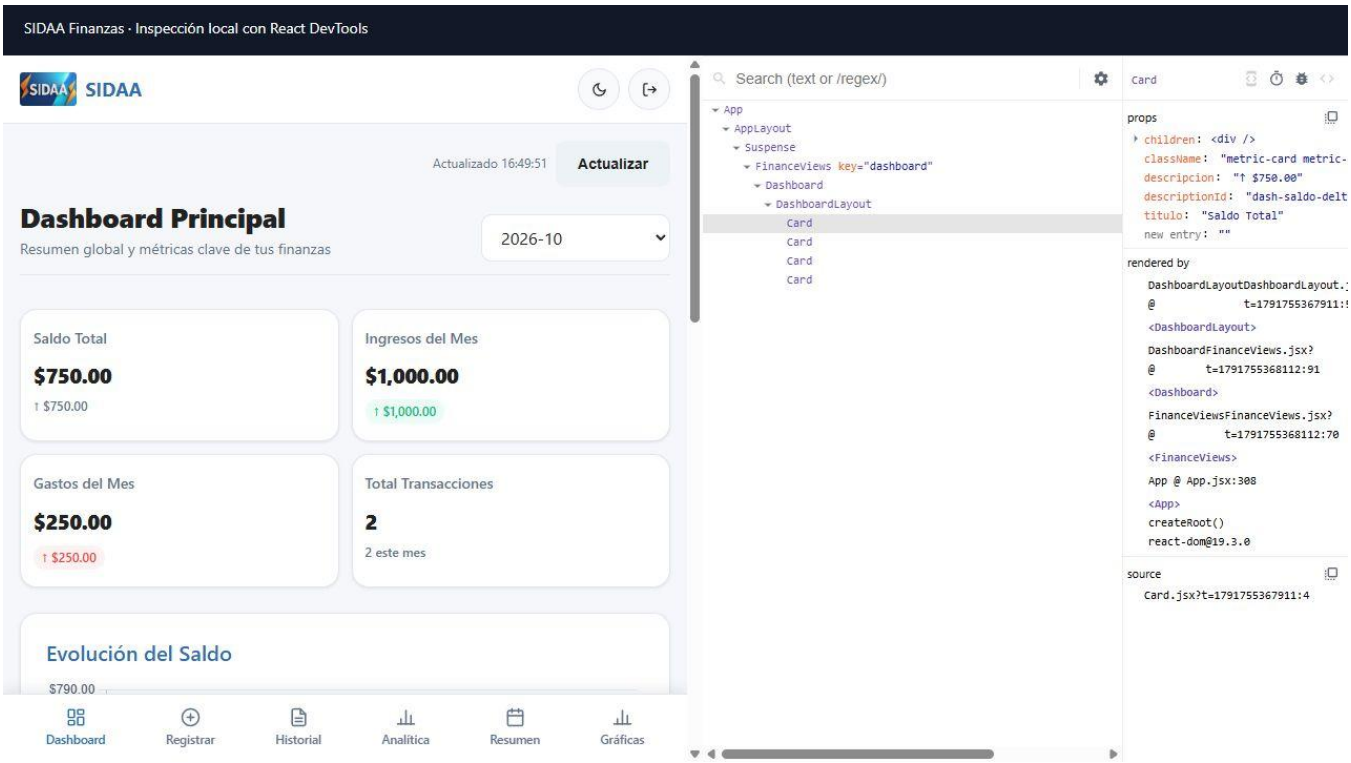
Las capturas de React DevTools que siguen ya formaban parte del repositorio revisado. Son evidencia histórica de la implementación de las semanas 5 y 6, no capturas nuevas del 18/10/2026. La suite actual vuelve a verificar la lógica de efectos.

Figura 15. Estado del formulario en React DevTools



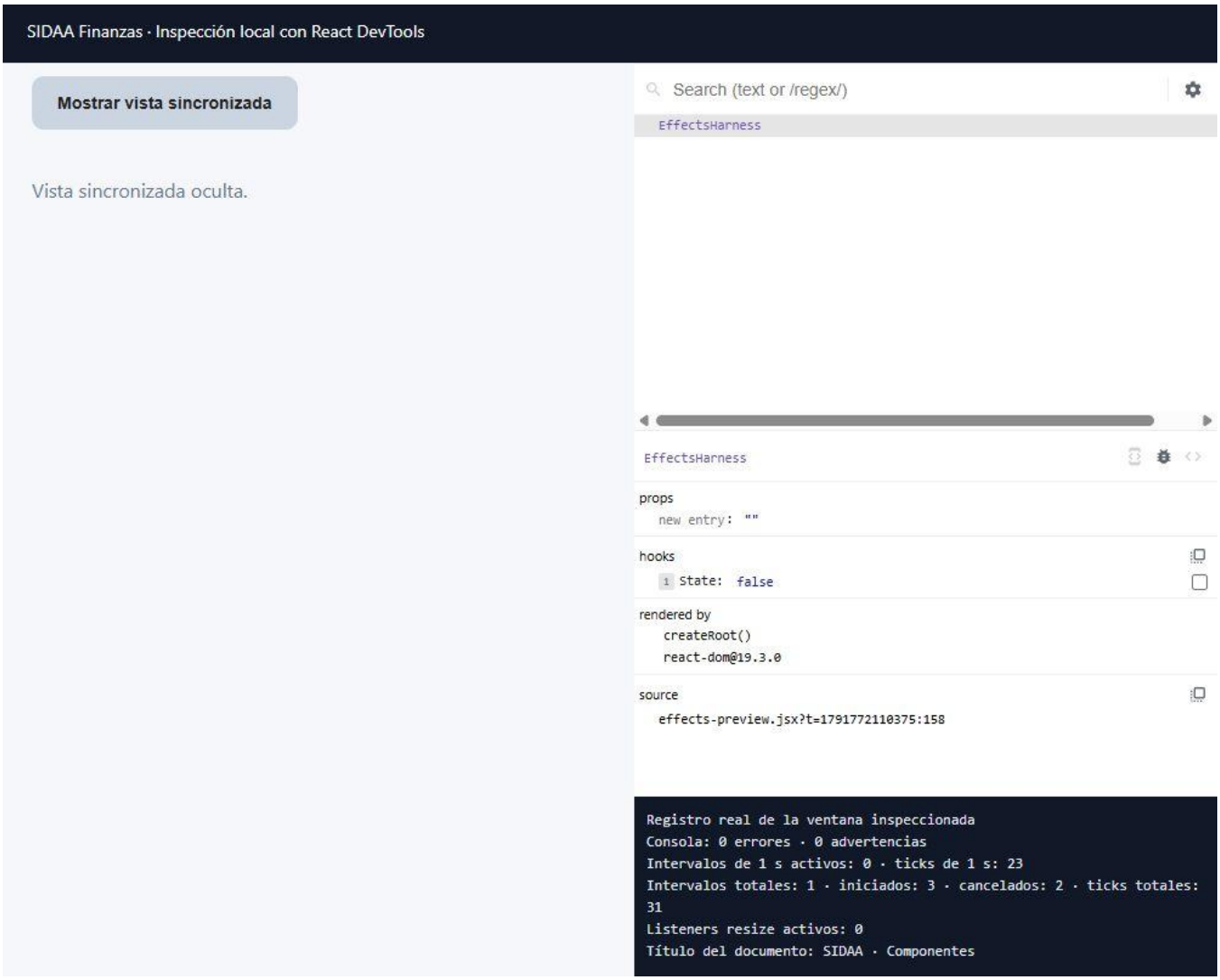
El estado tab selecciona form-register. Fuente: docs/evidence/react-devtools/02-crear-cuenta.jpg.

Figura 16. Props del componente Card



El inspector muestra las propiedades de una tarjeta financiera. Fuente: docs/evidence/react-devtools/05-card-saldo.jpg.

Figura 17. Limpieza de recursos al desmontar



Evidencia histórica del inspector local. La ejecución actual confirma la limpieza con las pruebas PA-01 a PA-06. Fuente: docs/evidence/useeffect/05-limpieza-completa.jpg.

Anexo 3 Registros de laboratorio y auditoría

Medición responsive en Render: en dashboard, historial y registro, los anchos de documento fueron 390, 768 y 1440 px para sus respectivos viewports. No hubo exceso de ancho global en las nueve comprobaciones. Esto no acredita todas las resoluciones ni sustituye pruebas táctiles físicas.

Consola del recorrido en Render: se registró el error de red HTTP 401 esperado al intentar una contraseña incorrecta; no se registraron excepciones JavaScript pageerror ni advertencias. En el laboratorio local anterior se provocaron HTTP 503 y HTTP 401 para verificar recuperación y sesión inválida. No se provocaron fallos de disponibilidad en producción.

Disponibilidad y persistencia de la ejecución documentada: raíz pública y /api/health respondieron 200 con environment=production. Los POST reales de registro y movimientos respondieron 201; una nueva sesión recuperó los mismos ids 11 y 12. Los dos DELETE respondieron 200 y el GET posterior devolvió []. El commit del código evaluado se conserva identificado; las actualizaciones posteriores organizan la documentación.

Protocolo de campo controlado: URL HTTPS de Render; una cuenta exclusiva; dos movimientos ficticios etiquetados; ninguna respuesta HTTP interceptada. Las pruebas comprueban el despliegue en internet, pero no constituyen un estudio con usuarios reales, dispositivos físicos o redes múltiples.

Identificación de ejecución: etiqueta S8-RENDER-1792305590956; registro real el 18/10/2026 a las 01:39:57; cierre a las 01:41:04, hora America/Bogota. La contraseña y el token se excluyen de las evidencias.

Trazabilidad de datos: id 11 = Ingreso, 1000.00 USD, 2026-10-18; id 12 = Gasto, 250.00 USD, 2026-10-18. La recarga y el contexto independiente devolvieron estos mismos valores. Después de eliminar ambos, GET /api/transactions devolvió un arreglo vacío. La permanencia entre sesiones demuestra almacenamiento del lado del servidor; no se inspeccionó directamente su base de datos.

Incidencia del instrumento: el primer recorrido se detuvo por una llamada incorrecta a status() en el arnés de pruebas Fetch. Se corrigió a la propiedad status y se reutilizó la cuenta ya creada. El registro PR-03 conserva el resultado real del primer intento. La incidencia no fue un fallo funcional de la aplicación.

Defecto visual medido en Render: el texto de ingresos del resumen ocupó 147 px dentro de un elemento de 105 px a un viewport de 1280 × 900. El valor numérico seguía siendo $1,000.00. La ausencia de desbordamiento global no descarta recortes dentro de las tarjetas.

Reproducción de Lighthouse: npx lighthouse https://finanzasmainupgraded.onrender.com/ --output=html --output=json --output-path=./lighthouse --chrome-flags="--headless=new". El informe usa Lighthouse 13.5.0 y el perfil móvil por defecto registrado en el apartado 6.

Archivos completos de auditoría y pruebas: disponibles en docs/examen/evidencias del repositorio. El PDF incluye adjuntos electrónicos con Lighthouse HTML y JSON, el registro de pruebas de Render y la guía Word de instalación. Los resultados, procedimientos y capturas se leen en el cuerpo del informe. El DOCX conserva además los registros históricos en su carpeta interna evidencias.

Anexo 4 Video grupal publicado

https://youtu.be/FHAjr41D-Qc
La exposición recorre el alcance del producto, componentes y estado, interfaz y accesibilidad, operaciones financieras, persistencia y API, pruebas, Lighthouse y consulta externa. Se muestran capturas reales del producto y evidencias del desarrollo. Los aportes se identifican a partir de la presentación inicial del equipo, sin atribuir implementaciones futuras al MVP actual.

Anexo 5 Instalación y ejecución local

Esta guía permite ejecutar en el equipo la aplicación React y su API, usando la versión evaluada del proyecto. Los comandos se ejecutan desde la raíz del repositorio en PowerShell o una terminal equivalente.

1 Requisitos y versión evaluada

Se necesita Git, Node.js 22.12 o superior de la rama 22, o Node.js 24, con npm; conexión a Internet para instalar dependencias; y una base PostgreSQL de pruebas ya creada, accesible y con SSL. La base puede estar en la nube aunque la aplicación se ejecute localmente.
Repositorio: https://github.com/ramko09-oss/finanzasmainupgraded
Rama de origen: main
`Commit evaluado: f736e2eaf5acad9194d0135eb8b2261534ee3139`

2 Descargar el proyecto

```
git clone https://github.com/ramko09-oss/finanzasmainupgraded.git
cd finanzasmainupgraded
git checkout f736e2eaf5acad9194d0135eb8b2261534ee3139
```

El último comando fija la versión evaluada. Mantenga esta carpeta como directorio de trabajo para los siguientes pasos.

3 Configurar la conexión y la sesión

Cree un archivo llamado .env en la raíz, junto a package.json. Copie este contenido y sustituya los valores de ejemplo por los de su base de pruebas. Use el puerto indicado por su proveedor.

```
PORT=3001
NODE_ENV=development
DB_TYPE=postgres
DB_HOST=HOST_POSTGRES
DB_PORT=5432
DB_USER=USUARIO_POSTGRES
DB_PASSWORD=CONTRASENA_POSTGRES
DB_NAME=BASE_PRUEBA
DB_SSL=true
JWT_SECRET=SECRETO_ALEATORIO_PROPIO
```

Para generar JWT_SECRET, ejecute el comando siguiente y copie su resultado como valor. Conserve .env solo en el equipo; no lo incluya en el informe ni en Git.

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

4 Instalar y preparar la aplicación

```
npm run build
```

Este comando instala las dependencias del backend y del frontend, y genera la compilación de React. No hace falta instalar por separado cada carpeta.

Para preparar las tablas en una base de pruebas nueva, ejecute:

```
node backend/init_postgres_db.js
```

El script crea o verifica las tablas usuarios y transacciones. No crea la base de datos: esta debe existir previamente. La versión evaluada del inicializador requiere una conexión PostgreSQL con SSL.

5 Iniciar y comprobar el funcionamiento

```
npm start
```

Mantenga esa terminal abierta y visite http://localhost:3001. Express sirve la aplicación React y la API desde el mismo puerto.

Compruebe http://localhost:3001/api/health: debe devolver status con valor ok. Después cree una cuenta de prueba, registre un ingreso y un gasto, y vuelva a iniciar sesión para comprobar que los movimientos se recuperan. La respuesta de salud por sí sola no verifica la conexión a la base de datos.

Para detener el servidor, pulse Ctrl+C en la terminal.

6 Modo de desarrollo y comprobaciones opcionales

Para trabajar en la interfaz con actualización automática, mantenga el backend en ejecución y abra otra terminal en la raíz:

```
npm run dev:react
```

Abra la dirección que indique Vite, normalmente http://localhost:5173. Las solicitudes de la API se envían al backend en el puerto 3001.

Para ejecutar las comprobaciones incluidas en el proyecto:

```
npm test
npm --prefix sidan-frontend run lint
```

Si aparece un error

Revise la versión de Node.js con node --version y confirme que .env esté en la raíz, sin extensión .txt. Si falla la conexión, revise host, puerto, usuario, contraseña, nombre de la base y acceso SSL. Si el puerto 3001 está ocupado, detenga la otra instancia antes de iniciar el servidor.

Estas instrucciones corresponden al commit indicado. La ejecución completa requiere proporcionar una base de pruebas y sus credenciales; el repositorio no las incluye.